AI-Enabled Quality Gates in CI/CD: A Controlled Experiment with Automated Tests, LLM Review, and Human Decision-Making

João Mendonça[1*] and Tiago Valente[1]

[1*]2º Ciclo Engenharia Informática, Universidade da Beira Interior, Covilhã, Portugal.

*Corresponding author(s). E-mail(s): joao.tiago.mendonca@ubi.pt;
Contributing authors: tiago.leandro.valente@ubi.pt;

Abstract

Continuous Integration and Continuous Delivery pipelines commonly rely on automated tests and static checks to detect defects before software changes are accepted. However, traditional quality gates are limited by the explicit rules and test cases defined by developers. This project investigates whether a Large Language Model can complement a conventional test gate by performing semantic review of source code and test code inside a CI/CD pipeline.

A small Python calculator project was implemented with GitHub Actions, pytest, and an AI-based quality gate using the OpenAI API with GPT-4o mini. The pipeline evaluates each change using two automated gates: a Test Gate based on pytest and an AI Gate based on LLM analysis. A third Human Gate was added manually to compare automated decisions with human judgment. Six controlled cases were designed to represent different decision patterns, including correct implementations, functional defects, missing edge-case handling, incorrect tests, AI false positives, and AI false negatives.

The results show that the AI gate can identify risks not detected by tests, such as missing division-by-zero handling. However, it can also reject contextually acceptable code or approve code with subtle semantic defects, such as silent precision loss. The experiment demonstrates that AI-enabled quality gates are useful as advisory mechanisms, but they should not fully replace human review or well-designed automated tests. The strongest value appears when AI review, test automation, and human judgment are combined.

Keywords: CI/CD, quality gates, software quality assurance, automated testing, GitHub Actions, large language models, AI-assisted code review

1 Introduction and Motivation

Software quality assurance is a central activity in modern software engineering. As development teams move toward shorter release cycles, Continuous Integration and Continuous Delivery pipelines are used to provide fast feedback on code changes. GitHub Actions supports this style of development by allowing workflows to automatically build, test, and evaluate code after repository events such as pushes or pull requests. [1]

Traditional CI/CD quality gates normally depend on predefined test cases, build checks, static analysis tools, or manually configured rules. These mechanisms are useful because they are repeatable and objective, but they cannot reason beyond their programmed checks. If a test suite does not include a specific edge case, the test gate may pass even when a real defect exists.

This project addresses that limitation by adding an AI-based quality gate to a CI/CD workflow. The AI gate uses an LLM to analyze both the implementation and the tests. Instead of only executing assertions, it evaluates the code semantically, considering correctness, safety, test coverage, and potential risks.

The motivation of the project is not to build a production-ready deployment system. Instead, the goal is to create a controlled experiment that demonstrates how AI-assisted review can be integrated into a software engineering workflow and critically evaluated. The project focuses on decision analysis: when do automated tests and AI agree, when do they disagree, and how should a human reviewer interpret these results?

2 Background and State of the Art

2.1 Continuous Integration and Quality Gates

Continuous Integration is a development practice in which changes are regularly integrated into a shared repository and automatically checked. GitHub Actions provides a CI/CD platform that can automate workflows such as building, testing, and deployment directly from a GitHub repository. [2]

A quality gate is a checkpoint that evaluates whether a software change satisfies certain quality criteria before progressing further in the development process. Research on quality gates in delivery pipelines describes them as mechanisms intended to increase confidence in code changes, while also creating a trade-off between release speed and release reliability.

In this project, the quality gate is implemented as a GitHub Actions workflow. The workflow runs after each push or pull request to the main branch. It executes the test gate first and the AI gate second. The final pipeline result fails if either pytest fails or the AI gate rejects the change.

2.2 Automated Testing with pytest

The traditional test gate in this project uses pytest. Pytest is a Python testing framework designed to make tests readable and scalable, supporting simple unit tests as well as larger test suites.

In this project, pytest verifies the behavior of two calculator functions:

```
add(a, b)
divide(a, b)
```

The tests check normal addition, normal division, and in later cases, division by zero. This provides a simple but useful context for evaluating how test coverage affects quality gate behavior.

2.3 AI-Assisted Code Review

Large Language Models can analyze source code and natural language prompts. In this project, the AI gate sends the contents of `calculator.py` and `test_calculator.py` to GPT-4o mini using the OpenAI API. The model is instructed to return a strict JSON decision with:

- `decision`: APPROVE or REJECT
- `risk`: LOW or HIGH
- `reasons`: short explanations

The implementation uses JSON response formatting to improve reliability of machine-readable outputs. OpenAI documentation describes structured outputs and JSON-oriented responses as mechanisms for obtaining more reliable model-generated structured data. [3]

Unlike a traditional rule-based static analysis tool, the AI gate does not rely only on fixed patterns. It interprets the relationship between the implementation and the tests. This allows it to identify some issues that tests miss, but it also introduces uncertainty and possible incorrect judgments.

3 Limitations of Existing Approaches

Traditional automated quality gates have several limitations.

First, test gates only check what the tests explicitly cover. If the test suite omits an edge case, the pipeline may pass despite a real defect. This happened in Case 3, where pytest passed because the tests did not include division by zero, while the AI gate rejected the implementation.

Second, static or rule-based gates can be too rigid. They may reject code because it matches a risky pattern, even when the development context makes the risk acceptable. This was observed in Case 5, where the AI rejected `eval()` as unsafe, while the human reviewer approved it as a temporary controlled implementation in an early experimental context.

Third, automated gates do not fully understand project intent. For example, a test may incorrectly expect `divide(6, 2)` to return 4. A test gate correctly reports failure, but the failure is caused by the test, not the implementation. The AI gate approved the implementation in this case, but did not explicitly classify the test as faulty.

Fourth, human review is flexible but subjective and slower. Human reviewers can detect subtle defects, such as silent precision loss, but their decisions may vary depending on experience, assumptions, and project context.

This project therefore evaluates a hybrid model: tests provide objective execution-based evidence, the AI provides semantic review, and the human reviewer makes the final contextual decision.

4 Project Design and Implementation

4.1 Repository Structure

The final project contains the following main files:

```
calculator.py
test_calculator.py
ai_quality_gate.py
api_check.py
requirements.txt
results.csv
.github/workflows/quality-gate.yml
README.md
```

The calculator application is intentionally small. This makes it easier to isolate quality gate behavior and interpret each experimental case.

4.2 Calculator Implementation

The final version of `calculator.py` contains:

```
def add(a, b):
    return a+b

def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return int(a/b)
```

The final implementation intentionally preserves a semantic issue for experimental analysis purposes: `divide()` converts the result to `int`, which causes silent precision loss for non-integer division. For example, `divide(5, 2)` returns 2 instead of 2.5.

4.3 Test Gate

The final test file is:

```
from calculator import add, divide
import pytest

def test_add():
    assert add(2, 3) == 5

def test_divide():
```

```
    assert divide(6, 2) == 3
    assert isinstance(divide(6, 2), int)

def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(6, 0)
```

The test suite passes for the final code, but it does not test non-integer division. This omission is important because it demonstrates how automated tests can miss defects when test coverage is incomplete.

4.4 AI Gate

The AI gate is implemented in `ai_quality_gate.py`. It reads the calculator implementation and the test file, sends both to GPT-4o mini, and requests a strict JSON decision.

The system prompt asks the model to evaluate:

- correctness
- safety
- test coverage
- code risks

If the AI returns REJECT, the script exits with code 1, causing the GitHub Actions job to fail.

The AI gate therefore behaves as an automated review step inside the CI pipeline. However, the model's decision is still interpreted as advisory evidence rather than absolute truth.

4.5 GitHub Actions Workflow

The workflow is defined in `.github/workflows/quality-gate.yml`.

The workflow performs the following steps:

- Checkout repository.
- Set up Python 3.12.
- Install dependencies from `requirements.txt`.
- Run pytest as the Test Gate.
- Run `ai_quality_gate.py` as the AI Gate.
- Print a quality gate summary.
- Fail the final pipeline if either gate fails.

The decision rule is:

- If TestGate = FAIL or AI = REJECT, the pipeline result is FAIL.
- If TestGate = PASS and AI = APPROVE, the pipeline result is PASS.

This means the automated pipeline is conservative: one negative automated result is enough to fail the workflow.

However, an important practical clarification is that the pipeline does not block the Git commit itself. The commit is already stored in the repository after `git push`. The workflow acts as a post-commit feedback system, not as a strict pre-commit enforcement mechanism.

5 Experimental Methodology

The experiment uses six controlled cases. Each case modifies either the implementation or the tests, then records:

- TestGate result
- AI result
- Human decision
- Final decision
- Whether a human override occurred
- Actual defect or risk
- Evidence from local execution or GitHub Actions

The three decision sources are defined as follows.

The Test Gate is the pytest result. It can be PASS or FAIL.

The AI Gate is the GPT-4o mini decision. It can be APPROVE or REJECT.

The Human Gate is the manual reviewer's decision after observing the implementation, test results, and AI output. It can be APPROVE or REJECT.

There are eight possible combinations:

- TestGate: PASS or FAIL
- AI: APPROVE or REJECT
- Human: APPROVE or REJECT

This gives:

$$2^3 = 8 \tag{1}$$

The project does not force all eight combinations. Instead, it covers six realistic and useful decision patterns.

6 Experimental Cases

Case 1: Baseline Safe Implementation

The first case uses a correct baseline implementation with normal addition, division, and appropriate handling of division by zero.

Result:

- TestGate: PASS
- AI: APPROVE
- Human: APPROVE
- Final: APPROVE
- Actual: No defect

This case confirms that the pipeline works correctly for a safe implementation. Both automated gates agree with the human reviewer.

Case 2: Faulty Addition Implementation

In this case, the `add()` function is modified to perform subtraction:

```
def add(a, b):
    return a - b
```

Pytest fails because `add(2, 3)` returns -1 instead of 5. The AI also rejects the change because the implementation is semantically incorrect.

Result:

- TestGate: FAIL
- AI: REJECT
- Human: REJECT
- Final: REJECT
- Actual: Defect

This is a true positive for both automated gates. The defect is obvious and detected by both pytest and the AI gate.

Case 3: Missing Division-by-Zero Handling

In this case, `divide()` is changed to:

```
def divide(a, b):
    return a/b
```

The tests only check normal division, so pytest passes. However, the AI rejects the implementation because division by zero is not handled and the tests do not cover that edge case.

Result:

- TestGate: PASS
- AI: REJECT
- Human: REJECT
- Final: REJECT
- Actual: Potential risk

This case demonstrates the value of AI semantic review. The test suite misses the issue, but the AI gate identifies a safety and coverage risk.

Case 4: Incorrect Expected Value in Test

In this case, the implementation is correct, but the test is wrong:

```
assert divide(6, 2) == 4
```

Pytest fails because the correct result is 3. The AI approves the implementation and does not reject the project.

Result:

- TestGate: FAIL
- AI: APPROVE
- Human: REJECT
- Final: REJECT
- Actual: Test defect

This case shows that test failures must be interpreted carefully. A failing test does not always mean the implementation is wrong; the test itself may be incorrect.

Case 5: Dynamic Execution Using `eval()`

In this case, the calculator uses dynamic execution:

```
def _execute(operation, a, b):
    operation_map = {
        "add": "a+b",
        "divide": "a/b"
    }

    if operation not in operation_map:
        raise NotImplementedError(f"Operation 'operation' not supported")

    return eval(operation_map[operation])
```

Pytest passes, but the AI rejects the code because `eval()` is a known security risk. The human reviewer approves the case because it is an early-stage controlled experiment and not production deployment code.

Result:

- TestGate: PASS
- AI: REJECT
- Human: APPROVE
- Final: APPROVE
- Override: Yes
- Actual: Context-dependent risk, not an immediate defect

This case is an AI false positive in the context of the experiment. The AI correctly identifies a risky practice, but the human reviewer decides the risk is acceptable temporarily.

Case 6: Silent Precision Loss

In this case, `divide()` converts the result to an integer:

```
def divide(a, b):
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return int(a/b)
```

The tests pass because they only check `divide(6, 2)`, which returns 3. The AI approves the implementation. However, the human reviewer rejects it because `divide(5, 2)` returns 2 instead of 2.5.

Result:

- TestGate: PASS
- AI: APPROVE
- Human: REJECT
- Final: REJECT
- Actual: Defect

This case is the most important failure scenario. Both automated gates miss a real issue, and only the human reviewer detects it. It demonstrates that AI review does not eliminate the need for human oversight or better test design.

7 Results

The results were recorded in `results.csv`.

Table 1 Summary of experimental case results

Case	TestGate	AI	Human	Final	Override	Actual
1	PASS	APPROVE	APPROVE	APPROVE	No	No defect
2	FAIL	REJECT	REJECT	REJECT	No	Defect
3	PASS	REJECT	REJECT	REJECT	No	Potential risk
4	FAIL	APPROVE	REJECT	REJECT	No	Test defect
5	PASS	REJECT	APPROVE	APPROVE	Yes	Context-dependent risk
6	PASS	APPROVE	REJECT	REJECT	No	Defect

7.1 Quantitative Metrics

7.2 Definitions

An AI false positive occurs when the AI rejects a change that is considered acceptable after human review. In this project, Case 5 is an AI false positive because the AI rejected the use of `eval()`, while the human reviewer accepted it due to the controlled experimental context.

An AI false negative occurs when the AI approves a change that contains a real defect or unacceptable risk. In this project, Case 6 is an AI false negative because the AI approved the silent precision loss caused by `int(a/b)`.

Defect leakage occurs when a defect passes automated gates and is detected later. Case 6 is also an example of defect leakage because both pytest and the AI gate approved the change, but the human reviewer rejected it.

A human override occurs when the human decision differs from the automated pipeline result. Case 5 contains a human override because the pipeline failed due to AI rejection, but the human reviewer approved the change.

Table 2 Quantitative metrics from the six experimental cases

Metric	Value
Total cases	6
TestGate failures	2
TestGate passes	4
AI rejections	3
AI approvals	3
Human rejections	4
Human approvals	2
Final rejections	4
Final approvals	2
Human overrides	1
AI false positives	1
AI false negatives	1
Cases where tests missed an issue or risk	2
Cases where both automated gates missed an issue	1

8 Discussion

The results show that AI-enabled quality gates can provide value beyond traditional tests. In Case 3, pytest passed because the tests did not include division by zero. The AI gate rejected the change because it identified the missing safety behavior. This demonstrates that LLMs can help detect gaps in test coverage.

However, the results also show that AI gates are not fully reliable. In Case 6, the AI approved an implementation that silently loses precision. This is a semantic defect that should have been detected either by stronger tests or by deeper code review.

The AI was also conservative in Case 5. It rejected `eval()` because it is commonly unsafe. From a security engineering perspective, this warning is reasonable. However, the human reviewer considered the development context and approved it as temporary experimental code. This shows that AI gates may lack contextual flexibility unless the prompt includes detailed project-specific risk tolerance.

The human gate remains important because it can interpret both technical evidence and project context. In this experiment, the human reviewer made the final decision and corrected both types of AI weakness: excessive strictness in Case 5 and missed semantic defect in Case 6.

The best result is therefore not obtained by replacing tests or humans with AI. Instead, the strongest model is collaborative:

- pytest provides objective execution evidence;
- the AI gate provides semantic review and coverage feedback;
- the human reviewer resolves ambiguity and context-dependent decisions.

9 Threats to Validity and Limitations

This project has several limitations.

First, the experiment uses only six controlled cases. This is enough to demonstrate key decision patterns, but not enough to produce statistically generalizable conclusions.

Second, the system under test is a simple calculator. Real software systems have more complex architectures, dependencies, security concerns, and business rules.

Third, AI model decisions may vary. The project uses `temperature=0`, but LLM behavior can still be affected by model updates, API changes, prompt wording, and context formatting.

Fourth, the human decision is subjective. Another reviewer might classify Case 5 differently and reject `eval()` even in an early-stage context.

Fifth, the pipeline is post-commit. It does not prevent the commit from entering the repository. It only reports quality status after the push.

Sixth, the AI gate depends on the OpenAI API and a valid API key. Billing, rate limits, quota errors, or network failures can affect reproducibility.

Seventh, the AI gate only reads `calculator.py` and `test_calculator.py`. Larger projects would require better file selection and context management.

10 Possible Improvements

Several improvements could make the project stronger.

The test suite could include more edge cases, especially non-integer division:

```
def test_divide_fractional_result():
    assert divide(5, 2) == 2.5
```

This would detect the defect in Case 6 automatically.

The AI gate could also enforce a stricter schema using modern structured output features instead of only JSON mode. OpenAI documentation notes that structured outputs can make model responses conform more reliably to a provided schema. [4]

The prompt could be improved by asking the AI to classify issues separately as:

- functional defect
- security risk
- test coverage gap
- maintainability concern
- context-dependent warning

Finally, the project could be extended to a larger codebase with multiple modules, more realistic tests, static analysis, mutation testing, and pull request comments generated by the AI.

Project Availability

The complete project source code, GitHub Actions workflow, and experimental cases are available at:

https://github.com/joaomendoncagit/ai-quality-gates

11 Conclusion

This project implemented and evaluated an AI-enabled quality gate inside a CI/CD workflow. The system combines pytest, GitHub Actions, GPT-4o mini, and manual review to analyze software changes from three perspectives: execution-based testing, semantic AI review, and human judgment.

The experiment shows that AI gates can detect risks missed by tests, especially when test coverage is incomplete. However, AI gates can also produce false positives and false negatives. The most significant case was silent precision loss, where both pytest and the AI approved the code, but the human reviewer rejected it.

The main conclusion is that AI-enabled quality gates are useful as support tools, not as complete replacements for testing or human review. Their value is highest when they are integrated into a broader quality assurance process where automated tests provide objective evidence, AI provides additional semantic analysis, and humans make final context-aware decisions.

The project therefore supports the idea that AI can improve software quality assurance, but only when used critically and collaboratively.

References

[1] GitHub Docs: Continuous integration with GitHub Actions. https://docs.github.com/en/actions/get-started/continuous-integration (2026)

[2] GitHub Docs: Understanding GitHub Actions. https://docs.github.com/en/actions/about-github-actions/understanding-github-actions (2026)

[3] pytest documentation: pytest: helps you write better programs. https://docs.pytest.org/ (2026)

[4] OpenAI: Structured model outputs. https://developers.openai.com/api/docs/guides/structured-outputs (2026)